

inline / inlineLambda / SRTP: Оптимизации в F#

Кривцов Илья, 24.44-мм

inline

- Компилятор встраивает байткод в вызывающую функцию напрямую, без генерации инструкции вызова
- В некоторых случаях поможет улучшить производительность приложения

inline

F# source

2 References

```
let add a : int b : int = // int -> int -> int  
| a + b
```

1 References

```
let addPlus42 a : int b : int = // int -> int -> int  
| add (add a b) 42
```

```
printfn "%d + %d + 42 = %d" 5 10 (addPlus42 5 10)
```

F# decompiled

```
[CompilationArgumentCounts(new int[] { 1, 1 })]
```

2 references

```
public static int add(int a, int b) =>  
    a + b;
```

```
[CompilationArgumentCounts(new int[] { 1, 1 })]
```

0 references

```
public static int addPlus42(int a, int b) =>  
    add(add(a, b), b: 42);
```

inline

F# source

2 References

```
let inline add a : ^a b : ^b = // ^a -> ^b -> 'c  
    a + b
```

1 References

```
let inline addPlus42 a : ^a b : ^b = // ^a -> ^b -> int  
    add (add a b) 42
```

```
printfn "%d + %d + 42 = %d" 5 10 (addPlus42 5 10)
```

F# decompiled

```
[CompilationArgumentCounts(new int[] { 1, 1 })]
```

0 references

```
public static c add<a, b, c>(a a, b b) =>  
    LanguagePrimitives.AdditionDynamic<a, b, c>(a, b);
```

```
[CompilationArgumentCounts(new int[] { 1, 1 })]
```

0 references

```
public static int addPlus42<a, b>(a a, b b) =>  
    LanguagePrimitives.AdditionDynamic<a, b, int>(a, b) + 42;
```

inline

- При излишнем использовании inline размер сборки может заметно увеличиться
- Стоит использовать inline с целью оптимизации только если это показывают замеры; не нужно на все функции навешивать inline (premature optimization)

inlinefLambda

- Если компилятор определяет, что аргумент вызываемой функции — лямбда-функция, то первая инлайнится в месте вызова, и туда же инлайнится лямбда-функция (“двойной инлайнинг”)
- Также может помочь с производительностью

inlineIfLambda

F# source

2 References

```
let mathOp op a : 'a b : 'b = // ('a -> 'b -> 'c) ->  
    op a b
```

1 References

```
let doMath a : int b : int = // int -> int -> int  
    mathOp (fun x : int y : int -> x + y)  
    (mathOp (fun x : int y : int -> x * y)  
        a b) 42
```

```
printfn "%d * %d + 42 = %d" 5 10 (doMath 5 10)
```

F# decompiled

```
[CompilationArgumentCounts(new int[] { 1, 1, 1 })]
```

2 references

```
public static c mathOp<a, b, c>(FSharpFunc op, a a, b b) =>  
    FSharpFunc<a, b>.InvokeFast(op, a, b);
```

```
[CompilationArgumentCounts(new int[] { 1, 1 })]
```

0 references

```
public static int doMath(int a, int b) =>  
    mathOp(doMath_32._instance,  
        mathOp(doMath_33_1._instance,  
            a, b), b: 42);
```

inlineIfLambda

F# source

2 References

```
let inline mathOp ([<InlineIfLambda>] op) a : 'a b : 'b =  
    op a b
```

1 References

```
let doMath a : int b : int = // int -> int -> int  
    mathOp (fun x : int y : int -> x + y)  
        (mathOp (fun x : int y : int -> x * y)  
            a b) 42
```

```
printfn "%d * %d + 42 = %d" 5 10 (doMath 5 10)
```

F# decompiled

```
[CompilationArgumentCounts(new int[] { 1, 1, 1 })]
```

0 references

```
public static c mathOp<a, b, c>(  
    [InlineIfLambda] FSharpFunc op, a a, b b) =>  
    FSharpFunc<a, b>.InvokeFast(op, a, b);
```

```
[CompilationArgumentCounts(new int[] { 1, 1 })]
```

0 references

```
public static int doMath(int a, int b)  
{  
    int num = a * b;  
    int num2 = 42;  
    int num3 = num;  
    int num4 = num2;  
    return num3 + num4;  
}
```

SRTP

- Statically Resolved Type Parameters
- В целом похож на generic, но вывод типов происходит во время компиляции
- Каждое использование функции с SRTP приводит к инстанцированию (похоже на C++)

SRTP

	Generic	SRTP
Время разрешения	Во время исполнения	Во время компиляции
Генерация кода	На один метод генерируется один байткод-метод	На каждую комбинацию типов инстанцируется генерируются байткод-методы
Использование на типах	Может быть использован на типах	Не может быть использован на типах
Использование с inline функциями	inline функция не может быть полностью generic	SRTP не может быть использован на не-inline функциях, то есть, функция обязательно должна быть inline

S RTP

- Компилятор определяет типы аргументов функции `sum` по её использованию в функциях `add_42i` и `add_42f`

4 References

```
let sum a : int b : int = // int -> int -> int  
  a + b
```

0 References

```
let add_42i a : int b : int = // int -> int -> int  
  sum (sum a b) 42
```

0 References

```
let add_42f a : int b : int = // int -> int -> int  
  sum (sum a b) 42.0
```

4 References

```
let sum a : float b : float = // float -> float -> float  
  a + b
```

0 References

```
let add_42f a : float b : float = // float -> float -> float  
  sum (sum a b) 42.0
```

0 References

```
let add_42i a : float b : float = // float -> float -> float  
  sum (sum a b) 42
```

SRTP

- При указании inline типы разрешаются локально, функция инлайнится

4 References

```
let inline sum a : ^a b : ^b = // ^a -> ^b -> 'c  
|  
  a + b
```

0 References

```
let add_42i a : int b : int = // int -> int -> int  
|  
  sum (sum a b) 42
```

0 References

```
let add_42f a : float b : float = // float -> float -> float  
|  
  sum (sum a b) 42.0
```

```
[CompilationArgumentCounts(new int[] { 1, 1 })]
```

0 references

```
public static int add_42i(int a, int b) =>  
  a + b + 42;
```

```
[CompilationArgumentCounts(new int[] { 1, 1 })]
```

0 references

```
public static double add_42f(double a, double b) =>  
  a + b + 42.0;
```

SRTP

- Можно обращаться к свойствам/методам объекта, не зная его тип

2 References

```
let inline getLength(object: 'a when 'a:(member Length: int)) = // ^a -> int  
|    object.Length
```

0 References

```
let getStringLength (s: string) = // string -> int  
|    getLength object = s
```

0 References

```
let getListLength (l: list<_>) = // list<'a> -> int  
|    getLength object = l
```

S RTP

- Можно обращаться к свойствам/методам объекта, не зная его тип

0 references

```
public static int getLength<a>(a @object)  
    => throw new NotSupportedException(message: "Dynamic invocation of get_Length is not supported");
```

0 references

```
public static int getLength(FSharpFunc<a, int> get_Length, a @object) => get_Length.Invoke(@object);
```

0 references

```
public static int getStringLength(string s) => s.Length;
```

0 references

```
public static int getListLength<a>(FSharpList<a> l) => l.Length;
```